

Project Documentation: 2-to-1 Parameterized Address Multiplexor

Digital Design Lab

Project Overview

This project implements a parameterized **2-to-1 Multiplexor** in Verilog HDL[cite: 3]. The multiplexor selects between two multi-bit input channels (`in0` and `in1`) based on a single select signal (`sel`) and routes the chosen value to the output (`mux_out`)[cite: 3].

It is designed for instruction and operand address routing with a default width of 5 bits[cite: 3].

Specifications & Functionality

- **Default Width:** `WIDTH = 5` (Parameterized)[cite: 3]
- **Selection Logic**[cite: 3]:
 - When `sel = 1'b0`: `mux_out` receives `in0` (Instruction Address)[cite: 3].
 - When `sel = 1'b1`: `mux_out` receives `in1` (Operand Address)[cite: 3].

Port Summary

Port Name	Direction	Type	Width	Description
<code>sel</code>	Input	wire	1	Select line[cite: 2, 3]
<code>in0</code>	Input	wire	<code>WIDTH</code>	Input data 0 (Instruction Address)[cite: 2, 3]
<code>in1</code>	Input	wire	<code>WIDTH</code>	Input data 1 (Operand Address)[cite: 2, 3]
<code>mux_out</code>	Output	reg	<code>WIDTH</code>	Selected multiplexor output[cite: 2, 3]

Table 1: Multiplexor Module Interface

Key Design Takeaway

Even though `mux_out` is declared as a `reg`, this design models **purely combinational logic** because it is driven inside an `always @(*)` block without a clock edge. No physical flip-flops or latches are synthesized.

Verilog Source Code & Explanations

Design Module (`multiplexor.v`)

```
module multiplexor #(
    parameter WIDTH = 5
) (
```

```

input  wire [WIDTH-1:0] in0,
input  wire [WIDTH-1:0] in1,
input  wire          sel,
output reg  [WIDTH-1:0] mux_out
);

always @(*) begin
    if (sel)
        mux_out = in1;
    else
        mux_out = in0;
end

endmodule

```

3.1.1 Code Detailed Breakdown

- parameter WIDTH = 5: Defines a configurable bit-width for the inputs and output, defaulted to 5 bits[cite: 3].
- always @(*): A combinational sensitivity list. The block evaluates whenever any input signal (sel, in0, or in1) changes state.
- if (sel) ... else ...: Implements the routing logic[cite: 3]. Blocking assignments (=) are used to guarantee standard combinational execution.

Testbench (multiplexor_test.v)

```

`timescale 1ns / 1ps

module multiplexor_test;

    localparam WIDTH = 5;

    reg          sel;
    reg [WIDTH-1:0] in0;
    reg [WIDTH-1:0] in1;
    wire [WIDTH-1:0] mux_out;

    // Instantiate UUT
    multiplexor #(
        .WIDTH(WIDTH)
    ) multiplexor_inst (
        .sel      (sel),
        .in0      (in0),
        .in1      (in1),
        .mux_out  (mux_out)
    );

    // Verification Task
    task expect;
        input [WIDTH-1:0] exp_out;
        begin
            if (mux_out !== exp_out) begin
                $display("TEST FAILED");
                $display("At time %0d sel=%b in0=%b in1=%b mux_out=%b",
                    $time, sel, in0, in1, mux_out);
            end
        end
    endtask

```

```

        $display("mux_out should be %b", exp_out);
    end else begin
        $display("At time %0d sel=%b in0=%b in1=%b, mux_out=%b"
            ,
            $time, sel, in0, in1, mux_out);
    end
end
endtask

// Stimulus Generation
initial begin
    sel = 0; in0 = 5'h15; in1 = 5'h00; #1 expect(5'h15);
    sel = 0; in0 = 5'h0A; in1 = 5'h00; #1 expect(5'h0A);
    sel = 1; in0 = 5'h00; in1 = 5'h15; #1 expect(5'h15);
    sel = 1; in0 = 5'h00; in1 = 5'h0A; #1 expect(5'h0A);
    $display("TEST PASSED");
    $finish;
end

endmodule

```

3.2.1 Testbench Highlights & Concepts

- **Case Equality Operator (!==):** Used inside the `expect` task to check for strict inequality, catching unexpected unknown (x) or high-impedance (z) states.
- **Custom Verification Task:** The reusable task `expect` modularizes output checking, printing formatted test results with the exact simulation timestamp (`$time`).
- **Delay Driven Verification:** Uses `#1` delays to give combinational logic time to update before checking results.

Simulation & Verification

Running with Synopsys VCS

To compile and simulate the design using Synopsys VCS in batch mode[cite: 3]:

```

vcs multiplexor.v multiplexor_test.v
./simv

```

Expected Output

```

At time 1 sel=0 in0=10101 in1=00000, mux_out=10101
At time 2 sel=0 in0=01010 in1=00000, mux_out=01010
At time 3 sel=1 in0=00000 in1=10101, mux_out=10101
At time 4 sel=1 in0=00000 in1=01010, mux_out=01010
TEST PASSED

```